

A Supervised Transformer-based Model for Attributing Mobile-App-generated Synthetic Audio Artifacts

Lucky Onyekwelu-Udoka and Yong Guan
Department of Electrical and Computer Engineering
Iowa State University
Ames, Iowa, USA
{lucky, guan}@iastate.edu

Abstract—The rising production and uses of synthetic audio in the online digital world, and various application domains have enabled many exciting applications. However, we also see the authenticity of audio artifacts (forensics) and their trustworthiness becoming severe problems in today’s rapidly evolving digital world. We (forensic communities) have found that more and more of such synthetic audio artifacts are generated using mobile apps. This paper focuses on detecting synthetic audio and attributing it to its sources (the apps or online services where it is generated), namely synthetic audio attribution. We present a transformer-based model for attributing such synthetic audio to its sources (e.g. mobile applications). We use the PatchOut Fast Spectrogram Transformer (PaSST) for pre-training and finetuning it to identify an audio artifact (deciding whether it is natural or generated using a synthetic approach/tool). For the detected fake (i.e. synthetic) audio artifacts, we try to attribute/link to their sources (mobile apps, for example VoiceAI, ClonyAI, Voice Changer, etc.). Through supervised learning, we have improved the model’s accuracy in audio classification and helped the understanding of the high-level features that the pretrained model extracted. We have trained and evaluated natural audio and four mobile application synthesizers. Our experimental results demonstrate that our model achieves 90% detection accuracy and 87% attribution accuracy for mobile synthesizers, using the pre-trained model’s feature extraction for complex audio artifacts.

Index Terms—Synthetic audio detection, audio forensics, PatchOut Fast Spectrogram Transformer (PaSST), deep learning.

I. INTRODUCTION

The rapid evolution of audio synthesis technologies, specifically mobile applications that can generate high-quality and semantically consistent audio for commercials, Text-to-Speech (TTS) systems, movies, etc., has significantly increased and transformed digital media [16][20][19].

Speech forensic recognition used for authenticating audio samples now faces new hurdles as these advanced synthesizers are testing the traditional methods designed to identify and verify speakers or detect tampering. The precision of modern TTS systems and other audio synthesis tools can mimic human speech nuances so closely that distinguishing synthetic audio from natural audio becomes increasingly tricky.

Although it offers creative opportunities to create content and entertainment, it raises serious concerns about misinformation, data security breaches, and copyright violations [13].

The people in New Hampshire were encouraged not to vote in the Democratic primary by a campaign that used a synthetic audio (Robocall) of US President Joe Biden during the election period in January 2024. This incident fueled fears of electoral disinformation and demonstrated how convincingly these technologies can mimic real individuals and potentially influence public opinion and electoral outcomes [18]. In addition, a deepfake showing Ukrainian President Volodymyr Zelenskii ordering his soldiers to lay down their arms and surrender the fight against Russia on 16 March 2022, which was immediately debunked, demonstrates the global implications of synthetic audio as it has the potential to influence public sentiment and international relations during a sensitive geopolitical conflict [3]. These instances show the dangers of synthetic audio and the urgent need for robust strategies to mitigate its impact.

The ease with which synthetic audio creation and distribution occurs in mobile applications challenges the authenticity and integrity of digital content, making audio verification and its sources essential in this digital age [31].

A challenging forensic problem due to the growing prevalence of synthetic audio is the issue of appropriately attributing synthetic audio to its source application and differentiating natural audio from synthetically generated audio. Existing methods for synthetic audio attribution detect only synthesizers or attribute them [6]. Many easily available audio synthesizers on mobile platforms/apps, each with varied degrees of realism and quality in audio production, make the task even more complex. In this research, we consider the following problem definition.

A. Problem Definition

Given an audio signal artifact $X \in \mathbb{R}^n$, where n is the length of the audio signal trace, the objective is to design a classification algorithm that outputs a classification label y , where:

$$y = \begin{cases} 0 & X \text{ is natural (i.e., human voice),} \\ i & X \text{ is generated by the } i\text{-th synthetic audio source,} \\ N+1 & X \text{ is synthetically-generated by an unknown source.} \end{cases}$$

where $i \in \{1, 2, \dots, N\}$ is one of the N known synthetic audio sources.

We formally define the problem targeted in this paper as follows

The objective function :

$$L(\theta) = -\frac{1}{n} \sum_{i=1}^n \sum_{j=0}^{N+1} y_{i,j} \log(f_j(x_i; \theta))$$

where: n : Total number of samples.

$f_j(x_i; \theta)$: The predicted probability that the input x_i belongs to class j .

$y_{i,j}$: Binary indicator (1 if the class is correct, 0 otherwise).

θ : Parameters of the model being optimized.

This objective function minimizes cross-entropy loss to optimize classification performance.

We propose an approach using a customized PaSST (Patchout Spectrogram Transformer) model to extract complex features from an audio signal trace. These features are then used to determine whether the audio trace is synthetically generated and, if so, attribute it to a specific source application (synthetic author generator).

Our main contributions in this paper include:

- 1) **Development of a Custom PaSST Model:** We design and train a custom PaSST transformer developed to extract features from audio signals.
- 2) **Classification for Detection and Attribution:** We fine-tune it to detect synthetic audio and accurately attribute it to the originating mobile application.

Our experimental results further validate our approach. We rigorously tested our model against a dataset that included both natural and synthetic audio from various sources, and it showed a remarkable ability to identify the source of synthetic audio and a superior performance compared to existing methodologies.

II. RELATED WORK

Audio synthesizing has altered the digital space through the advancement in artificial intelligence [26]. The generation of synthetic audio is primarily through Text-to-Speech(TTS) such as WaveNet etc., and voice conversion techniques such as Mellotron, etc. [29, 34, 17].

Given the potential for misuse in spreading misinformation or impersonating individuals, developing detection and attribution methods became necessary. Many synthetic audio methods have been proposed, such as the use of transform coefficients for synthetic audio detection, such as constant Q cepstral coefficients (CQCC) and gamma tone cepstral coefficients (GTCC) [23, 37].

In addition, early efforts relied on traditional machine learning, such as Mel-frequency cepstral coefficients (MFCC) and spectrograms, to classify audio signals. Recent deep learning methods use either the time domain of audio using the Recurrent Neural Network(RNN) or the Mel spectrogram representation of the audio using a neural network. Hua et al.[24] used an end-to-end deep neural network (DNN) to replace traditional CQCC and CQT with a model Time-domain Synthetic Speech Detection Net (TSSDNet), which uses ResNet or Inception-style structures. Bartusiak et al. [4] use the spectrogram representation to show the performance of transformer models for synthesized audio using different transformer architectures such as Compact Convolutional Transformer, Patchout faSt Spectrogram Transformer (PaSST) and Self-Supervised Audio Spectrogram Transformer (SSAST).

Several research studies focus on a narrow set of synthesized and synthetic audio types, leaving the diversity of audio that mobile applications can generate unexplored. Yadav et al. [21] explore the attribution of synthetic audio to its sources using a self-supervised learning approach with transformers. He used the Synthetic Speech Attribution Transformer (SSAT) model to process Mel spectrograms for attribution. Bhagtani et al. [5, 7] tackle the challenge of attributing synthetic audio to unknown synthesizers using an unsupervised method using PaSST transformers. Its model, the Fine-Grain Synthetic Speech Attribution Transformer (FGSSAT), employs dimensionality reduction and clustering to differentiate between synthesizers not seen during training. He also uses the Patchout Spectrogram Attribution Transformer (PSAT) to attribute synthesizers and distinguish new, unseen audio generation methods from those seen during training. Koutini et al. [28] proposed a method(PaSST) to reduce the computational complexity of transformer models that use Mel spectrogram patches for audio recognition, which helps to optimize and regularize transformers for audio spectrogram analysis.

Why Transformer Technology: Transformers represent a significant leap in handling complex tasks in audio attribution and detection, and through several research results [5, 7, 28, 4], they have outperformed traditional methods such as recurring neural networks (RNNs) in several key areas. One of the critical areas is its ability to manage long-range dependencies in data, which is crucial to understanding the full context of the audio inputs without the sequential processing limitations faced by RNNs. The self-attention mechanism within the transformer weighs each part regardless of its position in the sequence, allowing it to capture intricate relationships within audio data more effectively.

Transformer-based models such as the Patchout FaSt Spectrogram Transformer (PaSST) [5] and the Self-Supervised Audio Spectrogram Transformer (SSAST) [7] have set new benchmarks for performance. These models leverage the transformer architecture to achieve state-of-the-art synthetic audio detection and attribution results.

Our model uses the PaSST transformer, which sets it apart from existing methods. The PaSST handles feature extraction, while the classifier accurately attributes audio signals.

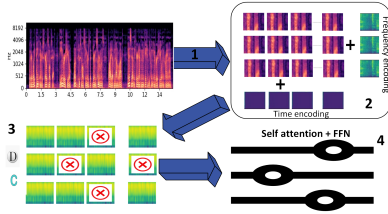


Fig. 1: PaSST architecture

III. METHODS

This section describes our proposed method in response to the need for technological advancement in synthetic audio detection and attribution. We use PaSST, as shown in Figure 1, renowned for its prowess in modeling sequential data to classify audio samples into natural or synthetic categories [28].

A. Data Preprocessing and Feature Extraction

Pre-processing is a crucial first step in our methodology, as we transform the raw audio signals into a representation that the PaSST can use. Since it is a one-dimensional temporal sequence, the audio is first converted into a 2D Mel spectrogram in the PaSST, offering a time-frequency analysis relevant to human hearing[35, 14, 36].

PaSST takes in an input audio signal of a fixed duration ($T = 10$ seconds). The input is [Batches, X], where X is the audio signal equal to the vector dimensional ($T \times S_R$), S_R is the sampling rate, and T is the duration of the audio. We used the Librosa library to load audio files with a consistent sample rate of 32KHz and to ensure that all audio clips were resampled to the rate. We truncate the signal for audio clips longer than T , while for shorter clips, we pad with zeros to ensure that the network receives input of a consistent shape.

We also introduced data-augmentation techniques using the PyDub library to extend our model's robustness and generalization capabilities. We applied pitch shifting, speed change, amplitude scaling, Gaussian noise, and time scaling in our audio dataset.

The audio data set is transformed to a Mel spectrogram in the PaSST using the *AugmentMeLSTFT* module, which extends the traditional Short-Time Fourier Transform (STFT) and applies a Mel scale filter bank to the frequency bins of the spectrum. The STFT is applied to the audio signal to convert it from the time domain to the frequency domain. It takes into account the time-varying nature of the signal. If we have a signal $x(t)$, the STFT is [2]:

$$STFT\{x(t)\}(\tau, \omega) = \int_{-\infty}^{+\infty} x(t) \cdot w(t - \tau) \cdot e^{-j\omega t} dt \quad (1)$$

Given $x(t)$ as the input signal, $w(t - \tau)$ as the window function centered at τ , ω as the angular frequency, and j as the imaginary unit.

This produces a two-dimensional signal representation that indicates how the frequency spectrum varies over time. The frequency bins are mapped to the mel scale using a mel filter

bank [2, 25]. The purpose of the Mel scale is to mimic the response of the human ear more closely than that of the linearly spaced frequency bands. Equation 2 is applied to get the mel frequency.

The mel frequency f_{mel} is related to the physical frequency f by the formula: [32]

$$f_{Mel} = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right) \quad (2)$$

The centered frequency $f(m)$ in hertz corresponds to the mel frequency m in Equation 3. Triangular filters are necessary for the MFCC calculation as they mark the midpoint of the peak response on the frequency axis [10].

$$f(m) = 700 \cdot (10^{m/2595} - 1). \quad (3)$$

Triangular filters $H_m(\omega)$ that look like triangles use this center to selectively capture spectral information, which is vital for tasks such as audio recognition and analysis tasks.

Each triangular filter $H_m(\omega)$ is defined as [10]:

$$H_m(\omega) = \begin{cases} 0 & \text{if } \omega < f(m-1), \\ \frac{\omega - f(m-1)}{f(m) - f(m-1)} & \text{if } f(m-1) \leq \omega \leq f(m), \\ 1 - \frac{\omega - f(m)}{f(m+1) - f(m)} & \text{if } f(m) \leq \omega \leq f(m+1), \\ 0 & \text{if } \omega > f(m+1). \end{cases} \quad (4)$$

The Mel spectrogram is computed by summing the energy in each triangular filter bank applied to the magnitude of the Short-Time Fourier Transform (STFT) of the signal[10]:

$$\text{Mel Spectrogram}(t, m) = \sum_{k=0}^{N-1} |X(t, \omega_k)|^2 \cdot H_m(\omega_k), \quad (5)$$

where N is the number of frequency bins in the STFT, $X(t, \omega_k)$ is the STFT magnitude at time t and frequency bin ω_k , and $H_m(\omega_k)$ is the value of the m -th mel filter in frequency bin ω_k . [10]

The Short-Time Fourier Transform (STFT) represents a time-frequency audio signal. The function $w(t - \tau)$ defines a window function that isolates the signal segments for analysis, allowing the model to capture the dynamic changes within the audio. The Mel spectrogram is preferred over a simple spectrogram because it emphasizes perceptual properties more relevant to human listeners and everyday audio processing tasks.

B. Patch Processing

Mel spectrogram with dimensions $H \times W$ (height H represents the frequency bins and width W represents the time steps) is divided into a series of patches of 16×16 since the transformer requires a sequence of tokens. The input audio waveform of duration t seconds is converted into a sequence of 128-dimensional log Mel filter-bank features using a 25ms Hamming window with a hop size of 5ms corresponds to win-length = 800 samples and hop size = 320 samples given a sampling rate of 32,000 Hertz. This results in a spectrogram

with dimensions 128×200 , which gives us the total number of nonoverlapping patches of 499 as computed using Equation(7) [15, 11]:

$$N = \left(\frac{H}{16}\right) \times \left(\frac{W}{16}\right) \quad (6)$$

The patches, a vector of size $16 \times 16 = 256$, are projected into an embedding space through a linear layer, creating a sequence of patch embedding [15]:

$$\text{Patch Embedding} = \text{Patch Vector} \cdot W_{\text{proj}} + b \quad (7)$$

where W_{proj} is the weight matrix of the linear projection layer and b is the bias vector.

This embedding is augmented with frequency and time positional encoding as it helps the transformer know the order of patches. Since the patches are derived from a 2D structure (spectrogram), the frequency and time positional encoding are important. The final embedding for each patch, E_i is [9, 11]:

$$E_i = \text{Patch Embedding}_i + \text{Frequency Pos Enc}_i + \text{Time Pos Enc}_i \quad (8)$$

Using patchout, which hides unused patches during training, the computational cost of the model is decreased from $O(n^2 \cdot d)$ for each layer to $O(\log n)$, where n is the sequence length (number of patches in this context) and d is the dimensionality of the model [28]. Each patch vector is linearly transformed to create a dense representation that is suitable for the model. This embedding incorporates positional encodings to preserve the temporal and frequency information essential for understanding the sequence.

Transforming the spectrogram into patches and embedding these allows the model to effectively process large-scale inputs and focus on local features within the context of the entire audio sequence. This method leverages the transformer's capability to handle data sequences, which is ideal for audio, where temporal and spectral dynamics are crucial.

C. Model Architecture

PaSST Transformer

The core of the PaSST Transformer is the self-attention mechanism that enables the model to weigh and integrate information from different segments of the audio signal irrespective of the distance within the sequence when making a prediction [25]. It helps capture the long-range temporal relationships within the audio data by computing attention scores for each patch embedding within each layer of a batch of patch embedding ($n \times e$), where n is the number of patches and e is the embedding size. Each of these patch embeddings is linearly transformed into three vectors: query (Q), key (K), and value (V). Each of Q, K, and V has a shape of [28]

$$n \times \frac{e}{h}, \quad (9)$$

where h is the number of attention heads.

The dot product computes the attention matrix A between the query of one patch and the key of all other patches $Q \times K^T$ [38]

$$A = \frac{QK^T}{\sqrt{e/h}} \quad (10)$$

where A represents the attention scores matrix, QK^T is the product of the dots of the query, and the transpose of the key matrix, and $d_k = \frac{e}{h}$ is the dimension of each key vector (or query). The division by $\sqrt{d_k}$ is the scaling factor that prevents the dot product from growing too large, as it helps the softmax function avoid regions where gradients are too small or large to learn from effectively.

After scaling, it passes through the softmax function to obtain the attention weights A [38]:

$$\text{Attention Weights} = \text{softmax}(A) \quad (11)$$

These attention weights compute a weighted sum of the Value (V) vectors, producing the output of the attention mechanism [38]:

A has a shape of $(n \times n)$

$$\text{Attention Output} = \text{Attention Weights} \times V \quad (12)$$

The self-attention layer's feedforward network uses a Rectified Linear Unit (ReLU) to process the attention-modulated data through two linear transformations, with activation applied in between. It helps the model learn complex relationships by introducing non-linearity to the model.

First linear transformation [15] 7 :

$$\text{Linear}_1(x) = W_1x + b_1 \quad (13)$$

W_1 is the weight matrix of the first linear layer, b_1 is the bias vector of the first linear layer, and x is the input feature vector.

ReLU Activation:

$$\text{ReLU}(z) = \max(0, z) \quad (14)$$

z is the output of the first linear transformation.

After the second linear transformation, the output of the feedforward network is combined with its input via a residual connection. This step tackles the problem of vanishing gradients and refines the features without losing the original information.

Second Linear Transformation:

$$\text{Linear}_2(y) = W_2y + b_2 \quad (15)$$

W_2 is the weight matrix and b_2 is the bias vector of the second linear layer, with y being the output of ReLU activation.

Each transformation's output combines with its input via a residual connection, enhancing feature preservation and tackling the vanishing-gradient problem.

Residual connection:

$$\text{Output} = x + \text{Linear}_2(\text{ReLU}(\text{Linear}_1(x))) \quad (16)$$

After being processed through multiple layers, the transformed features represent a rich contextual embedding of the

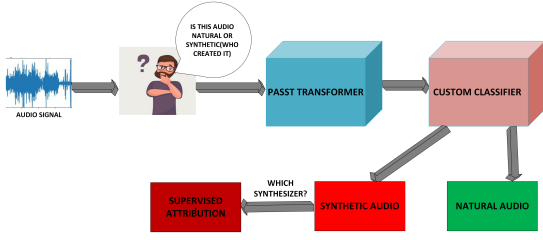


Fig. 2: PaSST, and our Proposed model for detection and attribution of audio signals

original audio input. These features can be used for various applications, such as unsupervised learning, other classification tasks, or content-based retrieval, depending on the needs of the downstream task. This flexibility is facilitated by removing the final classification layer, enabling the use of the model purely as a sophisticated feature extractor.

Proposed Model

Custom Model Configuration and Layer Freezing : We developed the custom PaSST to adapt the pre-trained PaSST model by removing and freezing some layers. This enabled the model to output raw features instead of class-specific predictions and provided a rich feature set for subsequent processing and classification. The CustomPaSST is a feature extractor that delivers a comprehensive representation of audio data that captures both spectral and temporal information of audio signals.

The decision to modify the PaSST model by removing its final layer and using it as a feature extractor is driven by several considerations:

Firstly, it allows our model to focus solely on extracting a rich and detailed feature set rather than performing direct classification.

Secondly, it uses the pre-trained capabilities of PaSST, which is optimized for audio processing.

Lastly, this modification increases its customizability and flexibility, enabling better fine-tuning of feature extraction to address specific challenges in synthetic audio detection and attribution.

```

1 model = get_basic_model(mode="logits")
2 model.net = get_model_passt(arch="
   passt_s_swa_p16_l28_ap476", n_classes
   =5)
3 model.net.head.dropout = nn.Dropout(p=0.5)

```

Program 1: Model Configuration

The model is initialized to output logits, which are raw, non-normalized predictions that we later convert to probabilities via a softmax layer during the training phase. This configuration initializes the model to output logits, with a specific dropout rate of 0.5 integrated into the classifier head. The choice of a dropout rate of 0.5 was finalized after experimenting

with other values, which did not yield as effective results in preventing overfitting. The dropout technique randomly deactivates a portion of the features during each training iteration, which is crucial to encouraging the development of a more robust and generalized model. The dropout is set to help prevent overfitting by randomly omitting features during training iterations, which encourages the model to develop more robust features.

Freezing Pre-Trained Layers: To leverage the robust pre-trained features while focusing on fine-tuning for specific audio classifications, we employ a strategic approach to freeze all layers except for the classifier head:

```

for param in model.parameters():
    param.requires_grad = False
for param in model.net.head.parameters():
    param.requires_grad = True

```

Program 2: Layer Freezing

This strategy involves freezing most of the network during training to preserve generalized feature detection capabilities acquired through prior extensive training on various audio datasets. In doing so, we maintain the integrity of the learned features while allowing the classifier head, which is directly responsible for the output of class predictions, to adapt to the specifics of our audio classification task.

The classification in the neural network is achieved through a softmax function applied to the logits, converting them into class probabilities. The following formula describes this process:

$$P(\text{class}_i) = \frac{e^{y_i}}{\sum_{j=1}^n e^{y_j}} \quad (17)$$

where y_i is the logit corresponding to class i , and n is the total number of classes (5 in this case). This softmax function normalizes the logits into probabilities that sum to one, making the output suitable for classification. [1]

This mathematical transformation is crucial for interpreting the raw output of the network as probabilities, allowing an accurate classification of audio into predefined categories.

IV. AUDIO DETECTION

Our detection methodology uses the features extracted by the CustomPaSST model, which captures spatial and temporal details from audio samples to determine the authenticity of the audio samples by identifying whether they are synthetic or natural.

Detection Process: During the detection process, audio samples are first pre-processed to align with the input requirements of the CustomPaSST model and then passed through the feature extraction phase. This phase processes the audio to assess the likelihood that the samples are synthetic. The output layer of the model employs a softmax function to convert the logits provided by the CustomPaSST model into probabilities.

These probabilities categorize each sample into synthetic or natural classes.

This approach of combining advanced feature extraction with effective classification through the softmax function ensures our model’s high accuracy and reliability. It makes the system a critical tool in the fight against audio-based misinformation and security breaches, effectively distinguishing between altered and authentic audio clips.

V. MOBILE APPLICATION ATTRIBUTION

Following detection, the attribution process seeks to determine the source of synthetic audio, such as distinguishing between different TTS systems or voice conversion tools and tracing it back to the applications that created them.

Attribution leverages the detailed feature set extracted by the CustomPaSST model. The classification task within this model is granular, with the aim of matching the audio signature to known profiles of synthetic mobile applications stored in our data set. This involves a more detailed analysis to correctly categorize the audio based on its characteristics.

Our proposed model attributes both closed and open set problems of audio signal in mobile applications. Given an audio signal X , the closed set problem attributes a synthetic audio signal to the known mobile application synthesizers it is trained to identify. X is tagged with a label y , which denotes its origin from a set of N mobile audio generators known to the system; for a given input X , our model using the PaSST can predict and attribute the label to one of the known classes[7].

$$y = f(X) \quad \text{where} \quad y \in \{1, 2, \dots, N\} \quad (18)$$

y denotes the label associated with X , indicating its origin from a set of N known mobile audio generators, and f represents the function modeled by the PaSST.

By addressing both closed- and open-set problems, our model can accurately attribute audio to known sources and adapt to detect new, unseen synthesizers.

VI. EXPERIMENTS

A. Training Dataset and Applications

We utilized a dataset that includes natural voice recordings and synthetic audio samples.

Dataset Composition and Mobile Synthesizers:

- 1) **Flickr Audio Dataset (Class 0):** often referred to as the Flickr8k Audio Caption Corpus, was created by researchers David Harwath and James Glass from the MIT Computer Science and Artificial Intelligence Laboratory as part of their work on exploring deep multi-modal semantic embedding for audio and images [27]. It contains 40,000 natural spoken captions for the pictures, which were collected using Amazon Mechanical Turk, an online platform that facilitates crowd-sourcing of tasks. All audios have an average duration of 5 seconds and are all sampled at 16KHz; they capture a wide range of speakers and offer variation in accent, phrasing, and vocal characteristics, which helps develop a robust system[22].

- 2) **ClonyAI (Class 1):** This mobile application has over 500k downloads and 9.76k reviews on both the Google Play Store and the Apple Store, with a rating of 4.5. It uses advanced artificial intelligence to provide voice and face cloning. It allows users to replicate any voice by uploading audio clips or direct recordings and transforming text to audio that closely mimics human beings. It also offers the FaceSync feature that animates still images to match voice inputs. It can create videos with synchronized lip movements and supports more than 20 languages, ensuring global accessibility [12].
- 3) **VoiceAI (Class 2):** This mobile application created by Neuron Labs leverages artificial intelligence to convert text into lifelike audio with a rating of 4.4 and over 15k reviews. It allows the generation of realistic audio from any text input, and it supports a wide range of uses, such as cloning one’s voice, targeting people of interest, creating music, and enhancing social media videos and podcasts. It offers an extensive selection of voice models, with new ones continually added, and it uses advanced audio analysis techniques to dissect critical elements such as pitch, tone, volume, and rhythm. Its output has a high resemblance to that of synthesized voices and individual natural voices that were cloned [30].
- 4) **Voice Changer (Class 3 and 4):** OnePixel Studio created this app. It has more than 500k downloads with a rating of 4.6 in the Apple Store and 4.2 in the Google Play Store, and it is versatile as it provides users with the capability to alter their voices with various entertaining effects. It has more than 25 audio effects, such as robot, alien, zombie, and squirrel; users can use it to record and modify their voices. It can also import audio files and create voices from text. For this experiment, we used both techniques. [39].

Labels are assigned to each audio file based on the application that generated it through scripts that parse the metadata to extract labeling information.

Data Analysis: We conducted a statistical analysis of the training dataset; each subset was analyzed separately to identify key characteristics and trends, including mean, variance, and standard deviation. The following is a detailed breakdown of the statistics and insights for each class in the data set.

Class-wise Data Analysis Summary: Flickr Audio Dataset (Class 0 - Real Audio)

- **Mean:** $[-7.25 \times 10^{-7}, 4.55 \times 10^{-3}, 6.44 \times 10^{-2}]$
- **Variance:** $[1.22 \times 10^{-8}, 8.15 \times 10^{-6}, 3.98 \times 10^{-4}]$
- **Std Dev:** $[0.00011, 0.00286, 0.01994]$
- **Insight:** Stable and consistent signals with minimal variability.

ClonyAI (Class 1)

- **Mean:** $[-1.54 \times 10^{-5}, 4.57 \times 10^{-3}, 6.18 \times 10^{-2}]$
- **Variance:** $[1.39 \times 10^{-8}, 1.13 \times 10^{-5}, 7.50 \times 10^{-4}]$
- **Std Dev:** $[0.00012, 0.00336, 0.02739]$

- **Insight:** Mimics natural voices with moderate signal variation.

VoiceAI (Class 2)

- **Mean:** $[8.96 \times 10^{-6}, 1.22 \times 10^{-2}, 8.92 \times 10^{-2}]$
- **Variance:** $[2.15 \times 10^{-7}, 2.86 \times 10^{-4}, 4.27 \times 10^{-3}]$
- **Std Dev:** $[0.00046, 0.0169, 0.0653]$
- **Insight:** High variability in tone and rhythm, suited for lifelike audio generation.

Voice Changer (Class 3 & 4)

- **Class 3 Mean:** $[-1.41 \times 10^{-5}, 8.34 \times 10^{-3}, 7.76 \times 10^{-2}]$
- **Class 4 Mean:** $[0.00172, 0.00720, 0.0689]$
- **Class 3 Std Dev:** $[0.00175, 0.0139, 0.0483]$
- **Class 4 Std Dev:** $[0.00888, 0.0155, 0.0495]$
- **Insight:** A wide range of effects creates highly varied outputs.

1) Overall Synthetic Audio Statistics:

- **Mean:** $[0.00041, 0.00777, 0.0732]$
- **Std Dev:** $[0.00450, 0.0133, 0.0491]$
- **Insight:** High variability due to multiple transformations.

This analysis shows key differences between real and synthetic audio in the dataset. Real audio is more stable and consistent, while synthetic audio introduces more variability in amplitude, pitch, and tone. This variability is essential for building models that generalize well across different types of audio.

Training Procedure

Optimizer Setup: We used the Adam optimizer with a learning rate of 0.0001 and a weight decay of 1×10^{-6} is implemented as a regularization measure to help prevent overfitting by penalizing larger weights in the model parameters.

We use the cross-entropy loss function, which is ideal for classification tasks, and a weighted random sampler is employed to manage class imbalance in the training dataset.

For our research, we generated a dataset comprising 300 samples for each application; the Voice changer application is capable of text-to-speech and voice conversion, and we generated a dataset from each technique.

With a focus on ensuring diversity across natural audio (class 0) and four synthetic audio classes(classes 1 to 4), we employed data augmentation techniques, doubling the dataset size to 1200 samples per application.

We used k-fold cross-validation in our training regimen. This method divides the data into subsets k , training the model in subsets $k - 1$ while using the remaining subset for validation. This cycle is repeated k times, and each subset is used exactly once for validation. We used K-stratified cross-validation, a necessity due to the limited size of our dataset. This method involved 7 folds and 35 epochs per fold, optimizing the balance between learning from the data and

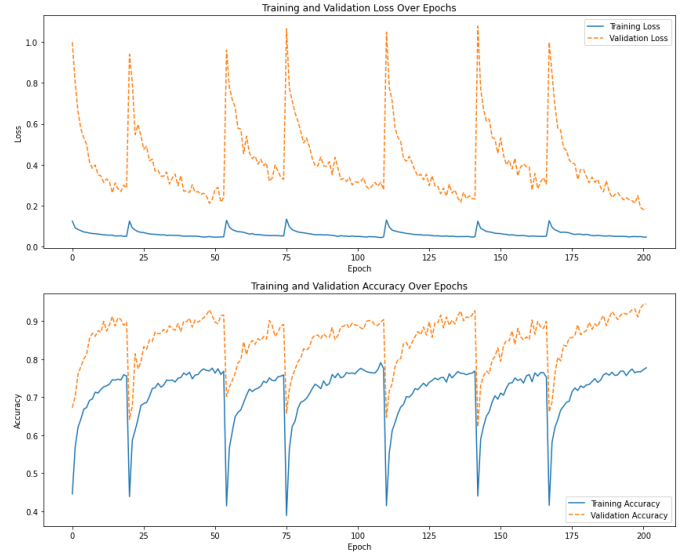


Fig. 3: Performance Graph

generalizing to new, unseen datasets.

A patience counter is an early stopping mechanism that is integrated to halt training if the validation loss does not improve for a predefined number of epochs (patience). This approach prevents unnecessary computations and potential overfitting by terminating the training process when additional training ceases to yield benefits.

Training Performance: Figure 2.2 reflects the results of implementing cross-validation of k-folds over 35 epochs for each fold.

The initial training loss starts at approximately 1.0, which is indicative of the model beginning with a high error at the start of training.

The final training loss decreases steadily to a level close to 0.2, showing significant improvement and stabilization in the learning capacity of the model.

The training accuracy begins around 40%, suggesting an initial difficulty in correctly classifying the training data, and is continuously improved to about 80%, demonstrating that the model has learned effectively from the training data.

B. Validation Performance

The validation loss fluctuates more significantly, with spikes reaching around 1.0 or higher at several points throughout the epochs. These spikes likely indicate the model's struggle with certain validation sets that could be more challenging or diverse compared to the training data.

Fluctuations in validation accuracy show notable variations with peaks and troughs, starting with a lower accuracy of 60% and having peaks that reach approximately 92% at certain epochs. The dips in validation accuracy typically correspond

to the spikes in validation loss, showing moments when the model’s generalization to new data is tested.

1) *Classification Report and Confusion Matrix*:: The classification report and the confusion matrix derived from these test data were invaluable. These tools provided a granular view of the model’s predictive performance across different audio classes. Specifically, the precision, recall, and F1 scores for each class were analyzed.

Class	Precision	Recall	F1-Score	Support
0 (Real Audio)	0.88	0.90	0.89	200
1 (ClonyAI)	0.93	0.94	0.88	192
2 (VoiceAI)	1.00	0.94	0.95	200
3 (VoiceChanger)	0.67	0.81	0.73	200
4 (VoiceT)	0.92	0.80	0.81	200
Overall	Accuracy		0.87	992
Macro Avg	0.88	0.87	0.85	992
Weighted Avg	0.88	0.87	0.85	992

TABLE I: Detailed performance metrics for each class and overall model performance.

Table 2.1 shows that

- **Class 0 (Real Audio)**: Shows strong performance with 88% precision and 90% recall, resulting in an F1 score of 89%. This indicates an effective identification of real audio.
- **Class 1 (ClonyAI)**: Shows very high precision at 93%, recall at 94% and F1 score of 88%.
- **Class 2 (VoiceAI)**: Outstanding performance with perfect precision (100%) and high recall (94%), leading to an impressive F1 score of 95%. This class shows the model’s excellent capability in identifying VoiceAI.
- **Class 3 (VoiceChanger)**: Struggles with precision at 67%, but high recall at 81% indicates the model can identify most VoiceChanger audio, although with many false positives. This is reflected in the lowest F1 score of 73% among all classes.
- **Class 4 (VoiceT)**: Very good precision at 92%, recall at 80% shows that some instances are missed, resulting in an F1 score of 81%. This indicates areas where the model’s performance could be enhanced.

For a detailed breakdown of the confusion matrix, including specific values for each class, please refer to Appendices A. These appendices contain the full classification report and the confusion matrices, respectively, offering a detailed and structured presentation of the model’s performance on the test dataset.

VII. DISCUSSION

Validation shows that the model accurately distinguishes between natural and synthetic audio. However, the attribution accuracy for synthetic classes is 87%. One of the significant challenges we encountered in the research was the model’s difficulty in differentiating classes 3 and 4, which are generated by the same application but different techniques, as

one is a text converter while the other is a voice changer. This similarity led to confusion in the model, which shows a limitation in the current feature extraction and classification methodology. The model’s inability to distinguish between these classes suggests that the features extracted by the PaSST model, while robust for identifying synthetic versus natural audio, may not capture the subtle nuances that differentiate audio samples generated by the same application under varied conditions.

In contrast to previous research, our research was conducted on a unique dataset for mobile app synthesizers. In contrast, previous studies, including the one by Bhagtani et al.[6], used the DARPA SemaFor Audio Attribution Dataset (SAAD) and the ASVSpooof2019 Dataset for their experiments. The results of this earlier research served as a benchmark, showing the innovative aspects of our approach. Although our model achieves an accuracy rate of 87%, this outcome shows the subtle challenges presented by our specialized data set compared to the more generalized and broader datasets used previously.

VIII. CONCLUSION

In this research, we proposed a method to detect and attribute synthetic audio generated by mobile applications. The proposed method uses a PaSST transformer integrated into a classifier using transfer learning: a set of features is extracted from the audio dataset, and a supervised classifier is trained to solve the classification problems based on the extracted features[8, 33].

Despite the promising directions shown by our research, several interests remain for further investigation. One of the critical challenges identified involves accurately attributing synthetic audio from the same source and across various applications within a closed-set scenario, incorporating additional features that capture subtler differences between classes. Our initial focus was on synthetic audio detection and attribution in contexts of logical access, omitting considerations such as the potential impact of noise, coding, or transmission errors and the complexities involved in attributing long-duration audio signals. Future studies will aim to expand on these preliminary findings. We are exploring these identified gaps and working towards more comprehensive solutions that can navigate the complexities of mobile application synthetic audio detection and attribution.

ACKNOWLEDGMENT

This work was partially supported by NIST CSAFE under Cooperative Agreements Nos. 70NANB15H176 and 70NANB20H019, NIFA No. 2021-67021-33775, and NSF under Grants Nos. CNS-1527579, CNS-1619201, CNS-1730275, CNS-2130889, AST-2232461, TI-2229654, DEB-1924178, ECCS-2030249, and by the Boeing Company. The findings and conclusions are those of the authors and do not necessarily reflect the views of the sponsors. We thank the anonymous reviewers for their valuable comments.

REFERENCES

- [1] Qingyao Ai et al. "Learning Groupwise Multivariate Scoring Functions Using Deep Neural Networks". In: *Proceedings of the 2019 ACM SIGIR International Conference on Theory of Information Retrieval*. ICTIR '19. Santa Clara, CA, USA: Association for Computing Machinery, 2019, pp. 85–92. ISBN: 9781450368810. DOI: 10.1145/3341981.3344218. URL: <https://doi.org/10.1145/3341981.3344218>.
- [2] J. Allen. "Short term spectral analysis, synthesis, and modification by discrete Fourier transform". In: *IEEE Transactions on Acoustics, Speech, and Signal Processing* 25.3 (1977), pp. 235–238. DOI: 10.1109/TASSP.1977.1162950.
- [3] Bobby Allyn. "Deepfake Video of Zelenskyy Could Be 'Tip of the Iceberg' in Info War, Experts Warn". In: *NPR* (Mar. 2022). URL: <https://www.npr.org/2022/03/16/1087062648/deepfake-video-zelenskyy-experts-war-manipulation-ukraine-russia>.
- [4] Emily R. Bartusiak and Edward J. Delp. "Synthesized Speech Detection Using Convolutional Transformer-Based Spectrogram Analysis". In: *2021 55th Asilomar Conference on Signals, Systems, and Computers*. IEEE, Oct. 2021. DOI: 10.1109/ieeeconf53345.2021.9723142. URL: <http://dx.doi.org/10.1109/IEEECONF53345.2021.9723142>.
- [5] K. Bhagtani et al. "FGSSAT: Unsupervised Fine-Grain Attribution of Unknown Speech Synthesizers Using Transformer Networks". In: *Proceedings of the IEEE Asilomar Conference on Signals, Systems, and Computers*. Pacific Grove, California, Oct. 2023.
- [6] Kratika Bhagtani et al. "Synthesized Speech Attribution Using The Patchout Spectrogram Attribution Transformer". In: *Proceedings of the 2023 ACM Workshop on Information Hiding and Multimedia Security*. IH&MMSec '23. Chicago, IL, USA: Association for Computing Machinery, 2023, pp. 157–162. ISBN: 9798400700545. DOI: 10.1145/3577163.3595112. URL: <https://doi.org/10.1145/3577163.3595112>.
- [7] Kratika Bhagtani et al. "Synthesized Speech Attribution Using The Patchout Spectrogram Attribution Transformer". In: *Proceedings of the 2023 ACM Workshop on Information Hiding and Multimedia Security*. IHMM-Sec '23. Chicago, IL, USA: Association for Computing Machinery, 2023, pp. 157–162. ISBN: 9798400700545. DOI: 10.1145/3577163.3595112. URL: <https://doi.org/10.1145/3577163.3595112>.
- [8] Clara Borrelli et al. "Synthetic speech detection through short-term and long-term prediction traces". In: *EURASIP Journal on Information Security* 2021.1 (2021), p. 2. ISSN: 2510-523X. DOI: 10.1186/s13635-021-00116-3. URL: <https://doi.org/10.1186/s13635-021-00116-3>.
- [9] Sam J. Callis. "Position Embeddings for Vision Transformers, Explained". In: *Medium* (Feb. 2024). URL: <https://towardsdatascience.com/position-embeddings-for-vision-transformers-explained-a6f9add341d5>.
- [10] Davide Carbone and Alessandro Licciardi. *Wavelet Scattering Transform for Bioacoustics: Application to Watkins Marine Mammal Sound Database*. 2024. arXiv: 2402.17775 [eess.SP].
- [11] Xiangxiang Chu et al. *Conditional Positional Encodings for Vision Transformers*. 2023. arXiv: 2102.10882 [cs.CV].
- [12] Clony AI. *Clony AI*. <https://www.clony.app/>. Accessed: insert date here. n.d.
- [13] Department of Homeland Security. *DEEP Increasing Threat of AKE Identities*. Available online. 2023. URL: https://www.dhs.gov/sites/default/files/publications/increasing_threats_of_deepfake_identities_0.pdf (visited on 03/27/2024).
- [14] K. Doshi. *Audio Deep Learning Made Simple: Automatic Speech Recognition (ASR), How It Works*. <https://towardsdatascience.com/audio-deep-learning-made-simple-automatic-speech-recognition-asr-how-it-works-716cfce4c706>. Accessed: [Your Access Date Here]. Jan. 2022.
- [15] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV].
- [16] Elevenlabs. *ElevenLabs - Generative AI Text to Speech and Voice Cloning*. Accessed Mar. 27, 2024. n.d. URL: <https://elevenlabs.io/> (visited on 03/27/2024).
- [17] A. Engler. *Fighting deepfakes when detection fails*. Brookings Institution, United States of America. Retrieved from <https://policycommons.net/artifacts/4139532/fighting-deepfakes-when-detection-fails/4947455/> on 30 Mar 2024. CID: 20.500.12592/wvgbcb. 2019.
- [18] France 24. "Biden Robocall: Audio Deepfake Fuels Election Disinformation Fears". In: *France 24* (Feb. 2024). URL: <https://www.france24.com/en/live-news/20240205-biden-robocall-audio-deepfake-fuels-election-disinformation-fears>.
- [19] Ghost. *Google's TTS AI: Elevating Speech Synthesis for Next-Gen Applications*. Accessed Mar. 27, 2024. Dec. 2023. URL: <https://blog.unrealspeech.com/googles-tts-ai-elevating-speech-synthesis-for-next-gen-applications/> (visited on 03/27/2024).
- [20] Ghost. *The 2024 TTS Overview: Harnessing AI for Advanced Speech Synthesis*. Accessed Mar. 27, 2024. Dec. 2023. URL: <https://blog.unrealspeech.com/the-2024-tts-overview-harnessing-ai-for-advanced-speech-synthesis/> (visited on 03/27/2024).
- [21] Yuan Gong et al. *SSAST: Self-Supervised Audio Spectrogram Transformer*. 2022. arXiv: 2110.09784 [cs.SD].
- [22] David Harwath and James Glass. *Deep Multimodal Semantic Embeddings for Speech and Images*. 2015. arXiv: 1511.03690 [cs.CV].
- [23] Farman Hassan and Ali Javed. "Voice Spoofing Countermeasure for Synthetic Speech Detection". In: *Pro-*

- ceedings of the International Conference on Artificial Intelligence. Settat, Morocco, Apr. 2021, pp. 209–212. DOI: 10.1109/ICAIS2203.2021.9445238.
- [24] Guang Hua, Andrew Beng Jin Teoh, and Haijian Zhang. “Towards End-to-End Synthetic Speech Detection”. In: *IEEE Signal Processing Letters* 28 (2021), pp. 1265–1269. ISSN: 1558-2361. DOI: 10.1109/lsp.2021.3089437. URL: <http://dx.doi.org/10.1109/LSP.2021.3089437>.
- [25] Rashmi Kethireddy et al. “Mel-Weighted Single Frequency Filtering Spectrogram for Dialect Identification”. In: *IEEE Access* PP (Aug. 2020), pp. 1–1. DOI: 10.1109/ACCESS.2020.3020506.
- [26] Z. Khanjani, G. Watson, and V. P. Janeja. “Audio deep-fakes: A survey”. In: *Frontiers in Big Data* 5 (2023). DOI: 10.3389/fdata.2022.1001063.
- [27] Jeffrey G. Klann and Peter Szolovits. “An intelligent listening framework for capturing encounter notes from a doctor-patient dialog”. In: *BMC Medical Informatics and Decision Making* 9.1 (2009), S3. ISSN: 1472-6947. DOI: 10.1186/1472-6947-9-S1-S3. URL: <https://doi.org/10.1186/1472-6947-9-S1-S3>.
- [28] Khaled Koutini et al. “Efficient Training of Audio Transformers with Patchout”. In: *Interspeech 2022*. interspeech2022. ISCA, Sept. 2022. DOI: 10.21437/interspeech.2022-227. URL: <http://dx.doi.org/10.21437/Interspeech.2022-227>.
- [29] Renqian Luo et al. “Lightspeech: Lightweight and Fast Text to Speech with Neural Architecture Search”. In: *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2021, pp. 5699–5703. DOI: 10.1109/ICASSP39728.2021.9414403.
- [30] Neutron Labs Inc. *Neutron Labs Inc. apps on the App Store*. <https://apps.apple.com/us/developer/neutron-labs-inc/id1610870504>. Accessed: insert date here. n.d.
- [31] NBC News. *Why AI-generated audio is so hard to detect*. Accessed Mar. 27, 2024. Feb. 2024. URL: <https://www.nbcnews.com/tech/misinformation/ai-generated-audio-detect-tool-model-rcna136634>.
- [32] Douglas O’Shaughnessy. *Speech Communication: Human and Machine*. Addison-Wesley, 1987, p. 150. ISBN: 978-0-201-16520-3.
- [33] Samuel Okozi et al. “Optimization of Energy using Vortex Coil Technology”. In: *International Journal of Electrical and Electronics Engineering* 10 (Aug. 2023), pp. 148–155. DOI: 10.14445/23488379/IJEEE-V10I8P114.
- [34] Yi Ren et al. “FastSpeech 2: Fast and High-Quality End-to-End Text to Speech”. In: *arXiv preprint arXiv:2006.04558* (2020). Submitted on 8 Jun 2020 (v1), last revised 8 Aug 2022 (this version, v8). DOI: 10.48550/arXiv.2006.04558. arXiv: 2006.04558 [eess.AS]. URL: <https://doi.org/10.48550/arXiv.2006.04558>.
- [35] RStudio. *A preprocessing layer to convert raw audio signals to Mel*. https://rdr.io/github/rstudio/keras/man/layer_mel_spectrogram.html. Accessed: 2024-03-26. 2024.
- [36] Muhammad Tanveer et al. “Mel-spectrogram and Deep CNN Based Representation Learning from Bio-Sonar Implementation on UAVs”. In: Jan. 2021, pp. 220–224. DOI: 10.1109/ICCCR49711.2021.9349416.
- [37] Massimiliano Todisco, Héctor Delgado, and Nicholas Evans. “Constant Q Cepstral Coefficients: A Spoofing Countermeasure for Automatic Speaker Verification”. In: *Computer Speech & Language* 45 (Sept. 2017), pp. 516–535. DOI: 10.1016/j.csl.2017.01.001. URL: <https://doi.org/10.1016/j.csl.2017.01.001>.
- [38] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL].
- [39] *Voice Changer - OnePixel Studio*. https://play.google.com/store/apps/details?id=studio.onepixel.voicechanger&hl=en_SG. Accessed: insert date here. n.d.

IX. APPENDIX A: CONFUSION MATRIX

The confusion matrix visually represents the accuracy of the model’s predictions across different classes, showing how many predictions were made correctly and how many were misclassified.

Figure 4 shows the confusion matrix analysis for the “Real vs. Fake Audio” classification task and demonstrates the model’s robust capability in differentiating between real and fake audio samples. The model successfully identified a significant proportion of both real and fake samples, achieving an overall accuracy of approximately 90.1%. Specifically, the model correctly classified 738 out of 792 fake audio samples and 191 out of 200 real audio samples.

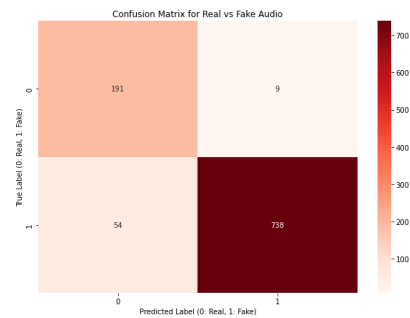


Fig. 4: Audio Detection

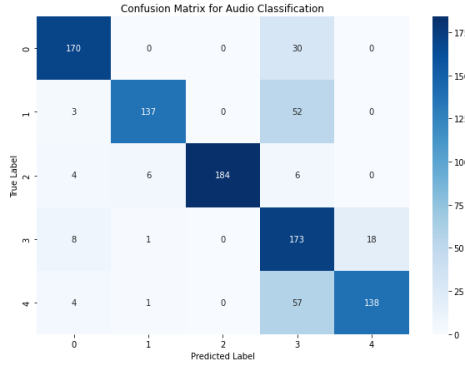


Fig. 5: Audio Attribution

Figure 5 The confusion matrix provided shows the model’s attribution performance across different audio categories, labeled from 0 to 4. Below is a detailed analysis of each label based on the matrix:

- **Label 0 (Real Audio):** The model correctly predicted the majority of real audio samples, with 170 correct predictions out of 200. There were 30 misclassifications where real audio was confused with Label 4, indicating a potential overlap in features or characteristics between these categories.
- **Label 1 (ClonyAI):** This category saw 137 correct classifications out of 192. There are a notable number of misclassifications, with 52 samples incorrectly identified as Label 3 (VoiceChanger), suggesting similarities or confusion between these synthesizer outputs.
- **Label 2 (VoiceAI):** Exhibiting strong performance, this label had 184 of 200 samples correctly classified. However, 6 samples were confused with label 1, which could suggest minor similarities in the synthetic characteristics exhibited by the two labels.
- **Label 3 (VoiceChanger):** Similar to Label 2 in terms of performance, VoiceChanger had 173 correct classifications of 200 out of 200. The primary confusion was with label 4, with 18 samples misclassified, suggesting possible overlaps in audio features between these two types.
- **Label 4 (VoiceT):** This label achieved reasonable accuracy with 138 correct predictions out of 200. The main issue was with Label 3, where 57 samples were incorrectly classified as VoiceChanger, highlighting significant confusion between these categories.

X. APPENDIX B: ABLATION STUDY ON THE IMPACT OF CLASS WEIGHTS AND DROPOUTS ON MODEL PERFORMANCE

This appendix details the ablation studies conducted to evaluate the impact of varying class weights and the application of dropout on the performance of our neural network model. These experiments were designed to understand how different configurations of class weights and dropout rates affect the model’s ability to generalize and perform across various classes of audio data.

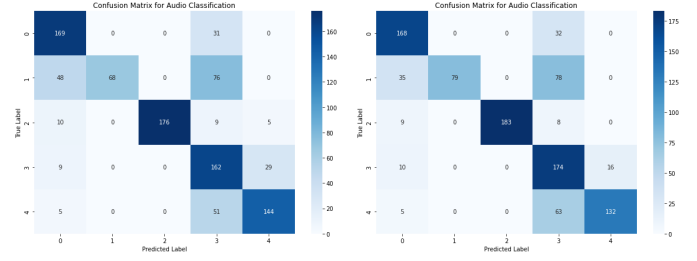


Fig. 6: [4.0, 7.5, 1.0, 0.5, Fig. 7: ([4.0, 7.5, 1.0, 0.5, 7.5])0.3
0.5

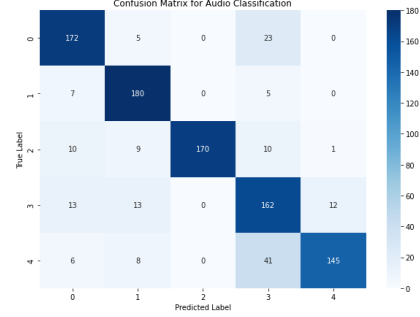


Fig. 8: ([4.0, 5.0, 1.0, 0.5, 5.0])0.4

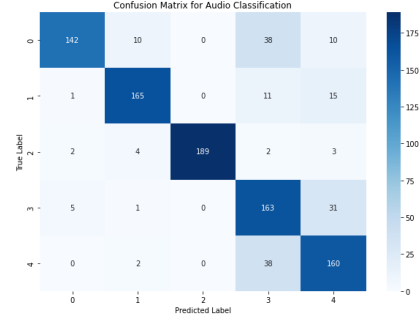


Fig. 9: ([9.0, 5.0, 1.0, 0.5, 9.0])0.45

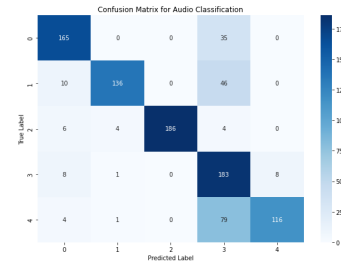


Fig. 10: ([3.0, 5.0, 1.0, 0.5, 3.0])0.3

Class Weights

Initially, the model was trained using a set of class weights specified as [3.0, 5.0, 1.0, 0.5, 3.0] for the classes in our data set. These weights were chosen based on the initial assessment of class imbalance, where classes with weights of 3.0 and 5.0 were identified as underrepresented.

Dropout

A dropout rate of 0.5 and 0.3 was randomly used, omitting half of the feature detectors in each training pass, thus forcing the network to learn more robust features.

Results and Discussion

Figures 6 to 10 show the results of these experiments. The visualizations show the confusion matrix under different configurations, showing how changes in class weights and dropout impact the model learning trajectory and generalization capability.

The adjustments in class weights had a noticeable effect on the model's ability to correctly classify underrepresented classes, improving recall for these classes without significantly compromising the precision. However, as shown in Figure 2.9 it was observed that excessively high weights did not consistently result in better recall. This anomaly could be attributed to the model's tendency to overfit to these heavily weighted classes, thereby impacting its ability to generalize across the broader dataset. Such outcomes show the importance of carefully calibrating class weights to avoid diminishing returns in model performance.

The implementation of a dropout rate of 0.5 helped mitigate model overfitting, particularly when class weights were adjusted aggressively. This strategic use of dropout ensured that the model remained sensitive to nuances in underrepresented classes while retaining its ability to generalize across the dataset. In addition, experiments with a dropout rate of 0.3 were carried out, which led to an increase in training accuracy but did not translate well to the test data. This suggests that while the lower dropout rate may allow the model to fit more closely to the training data, it potentially does so at the cost of reduced generalizability, as evidenced by the poorer performance on the test set.

The ablation studies conducted provide substantial evidence that class weights and dropout are integral to enhancing model performance in environments characterized by multiclass imbalances. Modifying class weights significantly influenced the model's capacity to identify underrepresented classes, improving recall effectively and accurately. However, excessively high weights did not consistently produce improvements, occasionally exacerbating overfitting. Similarly, the use of dropout has proven essential in curbing overfitting, with different rates showing varied impacts on model robustness and generalization. A lower dropout rate increased training accuracy but adversely affected performance in test data, underscoring the need for a balanced approach in its application.

These experiments show the complexity of model tuning in real-world scenarios and highlight that achieving optimal performance requires a nuanced understanding of how parameters like class weights and dropout interact with specific dataset characteristics and model architectures. The findings advocate for a cautious empirical approach to tuning these parameters,

suggesting that enhancements in model performance can be achieved by carefully balancing the trade-offs between fitting training data and maintaining the model's ability to generalize to new unseen datasets.